Defense presentation
Presenter notes

Erez Morabia · M.Sc. Advanced Project in Computer Science · 22997

Session: Zoom, Sunday, 10:00-11:00, as supplied by the student. The supervisor requests a clear explanation of problems in the current state of the art and how the project improves on them. No additional presentation requirements were specified. Plan approximately 20-25 minutes for the presentation within the one-hour meeting, leaving substantial time for discussion. English remains a working choice. The audience PDF contains 16 main slides and six backups with PDF outline navigation.

Executive-summary foundation

The main narrative follows the Executive Summary on pages 5-7 of PROJECT_DOCUMENTATION.pdf: the challenge, empirical organizational learning, three evidence signals, monthly adaptive blending, validation, results and pilot readiness. Architecture and the Black Pearl example come from sections 3.7 and 4. The contribution is the applied learning approach and working system. Transition evidence does not guarantee readiness. The report's 4-8 hours per month is a practitioner estimate, so the deck does not present it as measured savings.

Maturity assessments identify improvement gaps. This project addresses the practical decision: which practice should an individual team consider next? It uses longitudinal team and peer histories, observed practice transitions, and organization-wide trends to rank candidates. The working system and temporally controlled evaluation are the design contributions. The measured result is 58.0% conditional historical alignment, versus 30.6% random and 55.7% time-aware popularity. The small additional gain over popularity remains exploratory. The project did not compare implementations of the cited prior systems or test whether providing advice accelerates adoption.

Design basis

The deck follows university guidance on legibility and attention: a white background, dark navy text, one teal accent, embedded Arial fonts, 34 pt headings and predominantly 24 pt body text. The agreed narrative is why (slides 1-2), what (3-4), how (5-10), evidence (11-13), and implications (14-16). Visuals show inputs and outputs, parallel recommendation signals, a shared-engine architecture, and the software request. The later example outcome appears only in the evaluation section. These are selectable vector diagrams in the PDF. Diagram detail and supporting captions use smaller type. Small source footers and page numbers are supporting metadata. Quantitative graphics use direct labels, so meaning does not depend on color. This is a custom academic design, not an official Open University template.

MIT AeroAstro Communication Lab: Slide Design

Harvard T.H. Chan School of Public Health: Checklist: Improve Your Slide Design

Texas A&M Digital Accessibility: Accessible Presentations

Projection and rehearsal

On Zoom, share the PDF application window and keep these presenter notes on a separate screen. Use presentation or full-screen mode and check that the shared view shows slides only. Stop after slide 16 for questions. The six backup pages are A1 (monthly results), A2 (policy search grid), A3 (monthly policy selection) A4 (component normalization) A5 (raw evidence calculations) and A6 (code walkthrough). For a shorter talk, shorten algorithm and example details while preserving the problem statement, project response, metric definition and limits. Precompute any optional live backtest. Review your personal contribution before the session.

Timing

Aim for 20-25 minutes before questions. The 10:00-11:00 slot describes the whole meeting, rather than a requirement to speak for an hour. The demonstration is optional and would add approximately two minutes. Rehearse the first four slides as one connected argument: practical problem, empirical learning proposal, and recommendation approach.

01. Identifying Likely Large-Scale Agile Implementation Pathways

45 seconds. Open with the professor's requested framing. Existing assessment and recommendation approaches help organizations plan improvement, but this project addresses a particular operational decision: which practice should this team consider next? Explain the history-based ranking approach, the implemented decision aid and the backtest evidence. The project improves the organization's ability to make this decision systematically. Causal acceleration and superiority over published tools remain outside the demonstrated claims.

Sources: docs/PROJECT_DOCUMENTATION.md, title page, abstract and sections 1.1-1.4

02. The challenge: choosing the next practice

75 seconds. Define the practical limitation in the existing organizational process. Assessments identify many practices below maximum maturity, but the team still needs to decide where to invest limited coaching attention. Explain the three challenges on the slide: different starting maturity profiles, useful evidence in other teams' improvement histories, and the difficulty of comparing those histories manually at organizational scale. The goal is to turn that experience into two team-specific practices for coach review. This is the operational problem the project addresses. Avoid treating every existing maturity framework as a fixed universal roadmap. The next slide introduces the proposal directly: learning from organizational histories to guide the next-practice decision. No measured manual-analysis time savings are claimed.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 1.1-1.3 and 7.4

03. Learning organizational improvement behavior

90 seconds. Lead with the executive summary's central contribution: empirically learn how this organization improves, then use that evidence to identify likely next practices for one team. Collaborative filtering and transition modelling operationalize the approach. State the proposal before describing its implementation. Input is a team and prediction month, together with organizational history available at the earlier baseline. Output is two ranked practices that have not reached maximum maturity, when at least two candidates remain. The engine learns from prior organizational history using component-specific scoring windows. There is no fixed lookback for historical learning: each prediction uses only the history available under its baseline cutoffs. The dataset contains ten recorded months overall, not ten months of history for every prediction. The next slide notes explain the shorter component windows. The coach evaluates feasibility and context before deciding what to attempt. This slide specifies the decision aid rather than claiming a new algorithm or proven adoption acceleration. The next slide explains the recommendation idea conceptually, before the data and software architecture.

Sources: PROJECT_DOCUMENTATION.pdf, Executive Summary, pages 5-7; docs/PROJECT_DOCUMENTATION.md, sections 1.2, 3.5 and 4.3

04. Three signals form one ranking

105 seconds. Read the three evidence streams in parallel, not as sequential prerequisites. Similar teams contribute practices that comparable historical peer states later improved. Practice transitions contribute empirical successors to the target team's recently improved practices. Time-aware popularity contributes historical and recent organization-wide improvement counts. The arrows converge on component normalization and score blending. Monthly weights come from completed earlier outcomes. Only practices below maximum maturity enter the candidate set; the bottom ranking label does not mean candidate eligibility is decided after seeing the scores. The system returns exactly two practices when at least two candidates remain. Ranking scores are not calibrated success probabilities. Window details: collaborative filtering searches peer snapshots across all months strictly before the baseline, then measures improvements in each matched peer's next two recorded snapshots, stopping at the baseline. The Practice Transition Model learns patterns from all history strictly before the baseline; it identifies the target team's recent improvements by comparing the baseline against each of its two preceding snapshots. Time-aware popularity blends historical counts strictly before the baseline with the single latest organization-wide transition into the baseline. These shorter scoring windows do not cap the historical learning window. The three-snapshot forward window is for evaluating outcomes, not a scoring lookback. Snapshot counts need not equal calendar months. All signals remain bounded by the historical baseline. Define popularity as the frequency of recorded practice maturity increases across teams, rather than current high maturity or the number of teams using a practice. Historical counts summarize earlier organizational improvement events; recent counts summarize the latest observed interval. Each count vector is normalized under the component-specific rules before blending. A historical/recent balance is selected using completed earlier outcomes. The balance can assign all weight to history, as in the August worked example.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.3-3.5 and 6.5

05. Representing the organizational history

75 seconds. The report uses real organizational data from Avaya. Each observation is an assessed maturity profile, rather than an attempt log. Normalizing the raw levels makes component calculations consistent, but does not turn ordinal assessment levels into calibrated probabilities. Uneven observation coverage matters: a window of three recorded team snapshots is not necessarily three calendar months. One repeated team-month key resolves deterministically to its final source occurrence, leaving 654 analytical observations. Discuss detailed cleaning and missing-value questions using the project report. Distinguish units explicitly: 654 is the number of unique observed team-date snapshots, not the number of evaluated predictions. A prediction case is one team at one prediction date, supported by an earlier baseline and up to three later observations. For example, the July baseline and August, September and October outcomes support one Black Pearl August case; some of those snapshots also support other cases. The 298 cases are eligible May-September predictions with all three follow-up snapshots, not 298 different teams or 298 source rows. Source scores are 0-3 and normalized to 0-1. The observations come from one organization and do not record attempted practices. Explain the timeline: the first three recorded global snapshots, January, March and April, are reserved for initial history. Evaluation starts at the fourth snapshot, May, whose latest earlier assessment is April. This warm-up period is a design choice; we did not experimentally optimize its length. It is not a universal mathematical minimum for all components. February is absent from the source data. May through September are the five primary prediction months because their three-snapshot follow-up fits within the dataset ending in November. October and November are shown as remaining follow-up for that primary analysis; predictions for those two months are also reported separately as sensitivity cases with truncated global outcome windows. A snapshot can support multiple prediction cases.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.2, 5.5 and 6.1

06. High-level software architecture

75 seconds. This diagram follows section 4.1 of the project report, simplified for projection. The browser interface uses HTML, CSS and JavaScript and sends requests over HTTP/REST to the API layer. The API contains routes, request/response models and a service layer. Below it are three modules: the ML engine computes evidence and owns policy selection and ranking; validation replays historical predictions and computes metrics; the data module loads Excel, validates and normalizes maturity scores, and indexes histories. The upper arrows show API access to modules. The lower arrow from validation to ML explicitly shows that backtesting requests historical recommendations from the same ML engine, rather than maintaining an independent scoring implementation. The modules collaborate: validation uses the same recommendation engine and both consume processed history. Technical terms, if asked: PolicyEngine is the central scoring and policy-selection class. It selects the monthly settings from completed earlier outcomes and combines the three signals to rank eligible practices. RecommendationEngine is a thin compatibility wrapper: it exposes the recommendation interface and delegates the work to PolicyEngine, rather than implementing another model. The diagram uses responsibilities instead of these class names. Validation is where the BacktestEngine replays historical prediction cases, compares recommendations against later observed improvements and calculates ranking metrics, using the same scoring engine as the live application. The next slide explains those data dependencies and the shared scoring engine. CLI access is covered there. This slide shows the implemented modular structure and separation of responsibilities.

Sources: PROJECT_DOCUMENTATION.pdf, page 21, section 4.1; docs/PROJECT_DOCUMENTATION.md, sections 4.1-4.3

07. One shared engine serves the application

90 seconds. Read each row as a request and its returned result, not as independent data exports from PolicyEngine. In the top row, the user clicks Recommend; the API calls the shared recommendation engine with the selected team and prediction month. The engine returns two ranked practices, which are displayed in the browser. Both paths originate in the web interface and API: the upper branch is the Recommend action and the lower branch is the Run backtest action. In the bottom row, the API starts BacktestEngine, which loops over historical team-month cases and requests recommendations from the same engine. It then compares those results with later recorded improvements and calculates metrics. The two arrows in each row show logical call and response order; the result is returned to the caller. In the center, PolicyEngine has two responsibilities: once per prediction month it selects and caches global settings from completed earlier prediction outcomes, or uses the default popularity policy when none exist; for each team-month request it computes the evidence, applies the selected settings, ranks eligible practices and returns the top two. The 675-policy selection evaluates earlier cases and is distinct from the backtest outer loop over evaluated target months. Both live requests and historical replay consume the processed team histories introduced on the previous slides. The CLI also uses this engine but is omitted from the diagram for clarity. Backup A2 defines the grid and A3 explains monthly selection.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 4.1-4.3, 5.4, 6.5 and 7.3

08. One recommendation request, end to end

120 seconds. Explicitly introduce this as an explanation by example using a reproducible historical request. Explicitly identify Black Pearl as the team name and August 2020 as the prediction month. The question is which two practices this team should prioritize next. Find current state means locating the latest recorded assessment before that prediction: 5 July 2020, the baseline. The workbook encodes the August snapshot as 3 August 2020. Of the 30 retained practices, 28 are below raw maturity level 3 and eligible. There are 47 distinct qualifying peer teams with cosine similarity at least 0.75; the policy selects the 10 most similar. Each peer is represented by its best-matching historical snapshot strictly before the baseline, rather than necessarily its latest state. The selected 10 have similarities approximately 0.869-0.914. Weights are 25% collaborative-filtering similarity, 25% practice transitions and 50% popularity. The recent-popularity share is zero, so popularity is historical only. Policy selection uses all earlier prediction cohorts whose three-snapshot outcome windows have closed before August, not a fixed number of months back. Here only the 3 May prediction cohort qualifies: its outcome snapshots are 3 May, 8 June and 5 July. Thus selection relies on one completed prior prediction month, three calendar months before August; June and July predictions have not yet completed their outcome windows. This cohort selects the global policy across teams, rather than tuning it just for Black Pearl. Component learning separately uses the available historical evidence under its temporal cutoffs. After normalization and blending, the returned practices are Scrum Master at 0.750 and Tech debt strategy at 0.726. Scores rank candidates; they are not probabilities. Later improvements are reserved for the evaluation discussion on slide 12. slide 10 decomposes the scores.

Sources: docs/PROJECT_DOCUMENTATION.md, section 3.7; data/raw/combined_dataset.xlsx through the standard 90% missingness filter; src/ml/policy.py (case_components, completed_prior_months, select_policy, _selected_peer_indices); src/ml/similarity.py

09. Each component uses its own history

90 seconds. Separate component evidence from policy selection. For the August request, the baseline is 5 July. The transition model learns organizational patterns from all available snapshots strictly before July: January, March, April, May and June. It then identifies Black Pearl practices that improved when its July baseline is compared separately with its May and June snapshots. These are two comparisons, not two separate transition paths. A practice improved in either comparison enters one deduplicated set, and each practice contributes once as a trigger for learned successor patterns. Similarity matches the July target profile to peer states before July and observes up to two subsequent peer snapshots, never beyond July. Historical popularity also counts improvements strictly before July; the selected policy assigns zero share to the recent-popularity component. One cross-team policy is selected per prediction month and shared by all teams for that month; it is not fitted separately for Black Pearl. May only refers to selecting those global settings: replay recommendations for May across teams using the history available for those May cases, score candidate policies against May, June and July outcomes, and choose the best policy. August scoring then applies that policy to the richer history available for August. May is the first prediction cohort because the implementation starts at the fourth recorded global snapshot; January, March and April supply initial history. June and July prediction cohorts do not yet have complete outcomes before August. These are recorded-snapshot windows, not fixed calendar durations.

Sources: src/ml/policy.py: prediction_months, completed_prior_months, _peer_contribution, _month_popularity, _compute_components and select_policy; src/ml/sequences.py: learn_sequences_up_to_month

10. Why these two practices rank highest

105 seconds. Explain selection using only the evidence available for scoring, without revealing later outcomes. Both practices are eligible: Scrum Master is at raw level 2 and Tech debt strategy at raw level 1. Of 28 eligible candidates, these have the largest weighted totals. Scrum Master receives zero normalized similarity evidence, but the strongest transition and historical-popularity scores, both 1.000. Its score is 0.25 times 0 plus 0.25 times 1 plus 0.50 times 1 = 0.750. Zero similarity means no improvement contribution for this practice from the selected peers within the allowed windows; it does not mean no similar teams were found. Tech debt strategy receives support from all three components: 0.513 similarity, 0.525 transitions and 0.933 popularity. Their weighted contributions are approximately 0.128, 0.131 and 0.467, totaling 0.726 using unrounded inputs. A positive contribution from more components does not automatically yield a higher total. The model ranks numerical evidence; it does not prove suitability or predict a success probability. The component values shown are rounded and have component-specific normalization scopes. Later outcomes are discussed on slide 12. The main table now shows the numerator and denominator for every normalized component score, including DoD as the third-ranked eligible practice. Similarity raw values are 0, 0.3046660886028139 and 0.5943733751712366; divide each by the maximum 0.5943733751712366. Transition raw scores are 1.3333333333333333, 0.7 and 0; divide each by 1.3333333333333333. Historical popularity counts are 15, 14 and 12 improvement events, divided by the highest eligible-practice count of 15. Raw values in the slide are rounded; calculation uses full precision. The selected August weights are 25 percent similarity, 25 percent transitions and 50 percent historical popularity. Scrum Master total is 0.25 times 0 plus 0.25 times 1 plus 0.5 times 1, equal to 0.750. Tech debt strategy is approximately 0.25 times 0.513 plus 0.25 times 0.525 plus 0.5 times 0.933, equal to 0.726 with unrounded inputs. DoD is 0.25 times 1 plus 0.25 times 0 plus 0.5 times 0.8, equal to 0.650. It has the highest similarity score but lacks transition support and has lower popularity, so its overall rank is third. Similarity raw evidence sums each selected peer cosine times its best normalized maturity increase within the allowed follow-up. Transition raw evidence sums the learned top-three successor probabilities from each deduplicated recently improved target practice. Improvement counts may include multiple events across teams and time, not just distinct teams. Backup A4 elaborates the raw evidence and normalization scopes.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.5 and 3.7; data/raw/combined_dataset.xlsx; src/ml/policy.py: score_case and top_practices

11. Temporal controls prevent future-data leakage

120 seconds. Walk through the July-to-October date chart. The August prediction uses the most recent earlier team baseline, 5 July. Component learning precedes the baseline, and peer follow-up ends at the baseline. The August maturity values do not enter recommendation scoring. Policy selection uses only strictly earlier prediction months with completed outcomes. After generating the recommendations, evaluate against the August, September and October snapshots. Three recorded snapshots need not equal three calendar months because team observations can be irregular. These temporal controls separate scoring and policy selection from the later evaluation. Outcome conditioning still limits interpretation, as the next slide explains. Explain the documented rationale in report section 3.6: practice maturity improvements can take time to appear, so outcomes are checked across three recorded snapshots rather than only the first follow-up. Section 6.3 explains that policy selection waits until the complete outcome window of an earlier prediction month has closed; this prevents choosing settings using future outcomes. May is the first evaluated prediction month. Its outcomes are May, June and July, so they can first inform policy selection for August. Therefore May, June and July use the default bootstrap policy, 100% popularity with a 50/50 historical/recent split. The three bootstrap prediction months follow from the outcome-window rule; they were not selected independently. Distinguish this from the initial January/March/April history warm-up. The documentation supplies a qualitative adoption-timeline rationale, not an empirical comparison proving that three snapshots is optimal. Neither the warm-up length nor the outcome-window length was optimized. Do not claim a universal three-month adoption duration; the unit is recorded snapshots.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.6, 6.4 and 6.5; docs/flowcharts/global-three-factor-policy-blend.md

12. What counts as a successful ranking?

120 seconds. Revisit Black Pearl: Tech debt strategy improved in the August-to-October outcome window, so one of the two returned suggestions matches and the case counts as a hit. Define a hit as a nonempty intersection between the two recommendations and eligible practices that later improve. The reported rate conditions on at least one observed improvement. The stacked bar audits the stricter complete-team-window cohort: 120 improvement-bearing and 178 no-improvement cases out of 298. The primary hit rate on the next slide uses 121 outcome-bearing cases because one additional case has a partial team-level window. Do not combine the 121-case rate cohort with the 178 no-improvement cases as if they shared one completeness definition. Without records of attempted practices, the no-improvement cases cannot establish whether providing a recommendation would have changed outcomes. Average case hit indicators within each month, then average the monthly rates. Without improvement means that no eligible practice has a recorded maturity increase above baseline in any of the three later snapshots. Scores may remain unchanged or decrease. This does not establish that the team attempted and failed a practice or that a recommendation failed; the historical teams did not receive these recommendations, and attempted practices were not recorded. The partition is 120 plus 178 equals 298. The hit-rate cohort separately adds one improvement-bearing case with partial follow-up to reach 121.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.6, 6.2 and 6.3

13. Primary historical results

120 seconds. Read the chart in two comparisons. Candidate-aware random selection yields an expected 30.6 percent. The stronger time-aware-popularity arm yields 55.7 percent. The blend yields 58.0 percent. All three use the same monthly macro-aggregation convention. The random baseline accounts for each case's eligible candidate count and number of improvements. The popularity arm independently selects its recency setting from completed earlier outcomes. The 2.3 percentage-point gap is exploratory. It does not establish statistical superiority or an individual-team guarantee. Three of the five primary months use the same bootstrap policy in both arms. Pooled hits are 71 of 121, or 58.7 percent, but that is a different descriptive aggregation and is not the headline comparison. Explain the standalone comparison before reading the bars: time-aware popularity ranks eligible practices using only organizational improvement counts, balancing historical and recent evidence. It uses no peer similarity or practice-transition score. Its historical/recent balance is selected independently from completed prior outcomes under the same walk-forward rules as the adaptive blend. It is not necessarily the popularity component configuration chosen inside the blend. This controls for how much of the result can be achieved simply by recommending practices that commonly improved across the organization. Adjustable emphasis means that recent improvements do not automatically receive more weight. For example, a policy can combine 25% normalized historical score with 75% normalized recent score, giving recent evidence greater influence. The independent popularity comparison selects this weighting from completed earlier outcomes. In the Black Pearl August example, the popularity component inside the selected blend instead uses 100% historical and 0% recent. Avoid implying that this example necessarily shares the independently selected comparison policy. Recent refers to improvements over the latest recorded interval, not simply a practice having a recent assessment date.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 6.2-6.3

14. Capabilities, practical value and limits

105 seconds. Follow the executive summary from results to practical value. The functional prototype accepts Excel data and provides recommendations through a web interface and API. Monthly policy selection reduces manual parameter tuning, and a shared ranking approach supports consistent coach-team discussions. These are delivered capabilities. They do not constitute measured time savings or evidence of faster adoption. The short single-organization history and three identical bootstrap months limit performance interpretation. The system is ready for supervised pilot testing, while access control, monitoring and automated data refresh remain future engineering work. The next slide explains what that pilot should measure.

Sources: PROJECT_DOCUMENTATION.pdf, Executive Summary, pages 6-7; docs/PROJECT_DOCUMENTATION.md, sections 6.3 and 7.3

15. A prospective pilot can test practical benefit

75 seconds. Present this as a proposed next experiment, not an implemented study. If feasible, use randomized allocation, accounting for contamination through shared coaches. Record whether coaches actually consult the system and whether teams attempt recommended practices. Define outcome measures and analysis in advance. Include cases with no improvement, because the prospective question concerns the effect of offering recommendations across all enrolled cases. The pilot needs more than the current retrospective metric. It should measure time to improvement and unsuccessful attempts, while keeping assessment procedures comparable between groups. Exact sample size and duration require a separate study design.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.6, 7.3 and 8.2; allocation details are a proposed study design

16. Conclusions

45 seconds. Close with the takeaway: an empirical approach, learning from how teams improved, provides evidence for what another team could consider next. The approach is implemented in a working recommendation system, evaluated retrospectively, and ready for supervised pilot testing. Empirical means grounded in observed organizational improvement histories. Historical alignment does not establish faster adoption or a clear advantage over popularity alone. Stop here for questions; use the three appendix slides for monthly results, policy-search details or monthly policy selection when relevant.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 8.1-8.3

17. A1. Results by prediction month

The primary monthly macro-average uses exact rates before rounding. For the blend the five exact rates correspond to 6/21, 14/22, 18/27, 19/24 and 14/27. Their mean rounds to 58.0 percent. Popularity corresponds to 6/21, 14/22, 18/27, 18/24 and 12/27 and rounds to 55.7 percent. Pooled blend hits are 71/121=58.7 percent. Three primary months tie by construction through the bootstrap. The seven-month sensitivity cohort totals 151 outcome-bearing cases. Keep primary and sensitivity interpretations separate.

Sources: docs/PROJECT_DOCUMENTATION.md, section 6.3

18. A2. Policy search grid

Select the policy with the highest mean monthly Conditional Hit Rate@2 on completed earlier prediction months. Deterministic ties prefer more popularity, then lower recency, lower similarity weight, lower sequence weight, lower peer count and lower threshold. Candidate practice-score ties resolve by practice name. The blend uses independently normalized components. Bootstrap uses 100 percent popularity with equal historical/recent shares. Selection is automatic and global for each month. The grid is fixed in code. Temporal separation prevents future outcome leakage, but sparse earlier validation evidence still permits overfitting. A policy is a configuration of the scoring blend, not a separate model. Peer count is a maximum: after applying the minimum cosine threshold and deduplicating by team, take up to 5, 10 or 19 peers. A zero threshold imposes no positive similarity floor; 0.75 requires cosine similarity of at least 0.75. Component weights are ordered similarity, transitions and popularity. Each is 0, 25, 50, 75 or 100 percent; exactly 15 triples sum to 100 percent. For example, 25/25/50 gives popularity half the final score. Recent share is a separate inner weight within popularity: historical share is 100 percent minus recent share. Thus 25 percent recent means 75 percent historical; it is not a fourth outer component weight. The historical and recent inputs are normalized scores, not unscaled counts. Multiply the four option counts to obtain 675 configurations. They are evaluated on the same fixed outcome-bearing cases in completed prior prediction months; the selected policy maximizes mean monthly Conditional Hit Rate@2. The 5/10/19 values are the configured search choices; do not claim that their optimality was established. The component windows remain fixed at two recorded snapshots and are not searched.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.5 and 6.5

19. A3. One shared policy is selected each month

90 seconds. One global policy is selected for each prediction month, not separately for each team. The selection uses all earlier evaluated prediction months whose full three-snapshot outcome windows have closed strictly before the target month. Replay each of the 675 configurations using each earlier case historical components and candidate set, calculate Conditional Hit Rate@2 within each earlier month on the fixed outcome-bearing cohort, then average those monthly rates. Choose the maximum with deterministic tie-breaking. For August, only May predictions qualify because their May/June/July outcome snapshots are available before August. For September, May and June predictions qualify; June outcomes close in August. July predictions require September outcomes and cannot yet inform September settings. Apply the selected policy to target-month team evidence, so settings are shared but recommendations differ by team. Re-selection can return the same settings as before. With no completed earlier outcomes, use the fixed default of 100% popularity and equal historical/recent shares. The selection is cached per prediction month by PolicyEngine; it is not independently re-optimized for every request. Component learning still uses each case available history; May-only selection evidence does not restrict August component learning to May.

Sources: docs/PROJECT_DOCUMENTATION.md, sections 3.5, 3.7 and 6.5; src/ml/policy.py: completed_prior_months and select_policy

20. A4. How component scores are normalized

105 seconds. Reproduce the values with the checked-in workbook using the standard 90-percent missingness filter, July baseline and August policy. For similarity, select the ten peers above the threshold. For each peer and practice, find the best observed normalized maturity increase in up to two follow-up snapshots, ending no later than July. Sum peer cosine similarity times that increase. Tech debt strategy has raw evidence 0.3046660886028139; the largest similarity evidence across practices is DoD at 0.5943733751712366. Division gives 0.5125836743865602, displayed as 0.513. This is not the peer cosine itself. For transitions, collect recently improved target practices once, sum their learned top-three successor probabilities into each successor practice, then divide by the largest combined transition score. Tech debt strategy has 0.7 divided by 1.3333333333333333, giving 0.525; Scrum Master provides the maximum. Popularity counts observed improvement events strictly before July: Tech debt strategy has 14 and the largest count among eligible candidates is Scrum Master at 15, giving 0.9333333333333333. The August recent share is zero. Normalization scopes differ: similarity and transitions are normalized over all evidence before masking to eligible practices; historical popularity is first restricted to eligible practices and then normalized. Thus similarity denominator DoD can be outside the candidate set. Scrum Master has zero similarity evidence, and maximum transition and historical popularity evidence, producing components 0, 1 and 1. Blend the unrounded normalized values with 25/25/50 weights to get 0.750 and 0.726. Scores rank historical evidence and are not probabilities. Explain frequency versus weighted improvement using this illustrative example, not actual Black Pearl peer records: one peer with similarity 0.90 improves a practice from level 1 to level 3. Its normalized increase is two-thirds, so its contribution is 0.90 times two-thirds = 0.60. For another practice, two peers each have similarity 0.80 and each improves from level 1 to level 2. Each contributes 0.80 times one-third, about 0.267; together they contribute about 0.533. The first practice has one improvement event but a higher similarity evidence total than the second practice with two events. Thus DoD having the largest actual similarity total means its selected peers provide the strongest combined similarity-and-magnitude evidence, not necessarily the most frequent improvement events. For a peer, the implementation uses its best increase within the allowed two-snapshot follow-up, rather than counting every intermediate increase. In contrast, popularity counts recorded improvement events across all teams, ignoring peer similarity and increase size; a level 1-to-3 increase is still one event. Scrum Master has 15 such events, Tech debt strategy 14 and DoD 12 in the historical window. The popularity denominator 15 is the largest count among eligible practices, not the total across practices. What is the divisor in each row? Similarity: divide by the largest combined peer-improvement evidence total across practices. Here DoD supplies that maximum, 0.5944. Transitions: divide by the largest sum of learned follow-on probabilities across practices. Here Scrum Master supplies that maximum, 1.333. Historical popularity: divide by the highest improvement-event count among eligible practices, meaning the most frequently improved eligible practice. Here Scrum Master supplies the maximum of 15 events, so Tech debt strategy is 14 divided by 15. These divisors are separate maxima, not sums across practices, not the number of teams and not the number of observations. They are calculated for this case from its available evidence, rather than being fixed constants. The maximum becomes 1; if a component has no positive evidence, all its scores are zero.

Sources: data/raw/combined_dataset.xlsx; src/ml/policy.py: _peer_contribution, _compute_components, _normalize and score_case; docs/PROJECT_DOCUMENTATION.md, sections 3.5 and 3.7

21. A5. Where raw scores and divisors come from

Use the visible examples on A5 to explain where raw evidence comes from. Among the ten selected peers, only ACM improved Tech debt strategy in the allowed follow-up, from raw level 1 on 8 June to raw level 2 on 5 July. Its cosine similarity is 0.9139982658084418; multiply by the normalized one-level increase of one-third to obtain 0.3046660886028139, displayed as 0.3047. For transition evidence before July, Backlog grooming sprint has six recorded outgoing successor links, two to Scrum Master; Retro has one outgoing link, to Scrum Master. Because Black Pearl recently improved both practices, their 2/6 and 1/1 probabilities contribute to Scrum Master, totaling 1.3333333333333333. These are counts of practice-to-practice links between consecutive improvement-bearing steps, not necessarily distinct teams or consecutive calendar months. Popularity counts improvement events across earlier organizational histories: Scrum Master 15 and Tech debt strategy 14. A4 then divides these raw evidence values by component-specific maxima.

Sources: data/raw/combined_dataset.xlsx; src/ml/policy.py: _peer_contribution and _compute_components; src/ml/sequences.py: _learn_team_transitions and get_typical_next_practices

22. A6. Code walkthrough

Walk through PolicyEngine._compute_components(team, baseline_month), currently at src/ml/policy.py:335. It collects the baseline maturity levels, practices below maximum maturity, historical peer matches and per-practice improvements, target-team recent-improvement transition evidence, and historical/recent organizational popularity counts. It returns CaseComponents. Policy-specific peer selection, normalization and weighting occur later in score_case; final top-two selection occurs in top_practices. Also cover score_case(case, policy), currently at src/ml/policy.py:544. It selects the policy peer subset, combines similarity and improvement magnitudes, normalizes component evidence, mixes historical and recent popularity and applies the outer component weights. Its output maps each eligible practice to a blended score. It can also return component breakdowns. It does not select the monthly policy or choose the final top two; those responsibilities belong to select_policy and top_practices. Both PDF source pointers link to the repository source. Line numbers may move as the code changes.

Sources: src/ml/policy.py:335, PolicyEngine._compute_components